

Digital Signal Processing

Assignment #3

Haar-Wavelet Transform

소속: 컴퓨터정보공학부

학번: 2020202090

이름: 최민석

1. 과제 개요

이번 assignment#3에서는 입력 이미지에 대해 Haar-Wavelet 변환, 역 Haar-Wavelet 변환, 스칼라 양자화를 수행할 수 있는 프로그램을 구현한 후 이를 이용해 Haar-Wavelet 변환을 수행한 여부에 따라 달라지는 복원 이미지의 화질을 비교해볼 것이다.

화질 비교는 출력 이미지에 대한 MSE 를 계산하고 수치를 비교하는 객관적 비교와, 직접 이미지를 보고 화질을 비교하는 주관적 비교를 통해 수행한다.

데이터량은 각 대역의 레벨에 따라 양자화 레벨을 달리하여 적용한다. 인간이 인지하기 힘든 고주파 대역의 신호에 높은 양자화 레벨을 적용한다. 그러고나서 양자화율에 따른 데이터량과 신호 손실을 비교한다.

입력 이미지는 512x512, 256x256 해상도의 YUV4:0:0, 8-bit 의 .raw 이미지 파일 5 개가 주어진다. (프로젝트 내 input 디렉토리에 저장)

- Airplane_256x256_yuv400_8bit.raw
- Baboon_256x256_yuv400_8bit.raw
- Barbara_512x512_yuv400_8bit.raw
- Couple_512x512_yuv400_8bit.raw
- Lenna_256x256_yuv400_8bit.raw

주어진 이미지들에 대해 Haar-Wavelet 변환을 수행하고 양자화를 진행한 뒤 역 양자화를 수행하고 역 Haar-Wavelet 변환을 수행해 이미지를 합성하여 재구성한다. 양자화는 스칼라 유니폼 혹은 스칼라 논유니폼 양자화를 구현하여 사용한다. Haar-Wavelet 변환은 이미지를 필터링하여 저주파 성분과 고주파 성분으로 재귀적으로 분해한다.

2. 과제 수행 방법

- Haar-Wavelet 변환

```
void haarTransform(vector<vector<double>>& image, int rows, int cols)
{
    vector<vector<double>> temp(rows, vector<double>(cols));
    // horizontal transform
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols / 2; j++) {
            temp[i][j] = (image[i][2.* j] + image[i][2.* j + 1]) / sqrt(2.0); // low pass
            temp[i][j + cols / 2] = (image[i][2.* j] - image[i][2.* j + 1]) / sqrt(2.0); // high pass
        }
    }
    image = temp;

    // vertical transform
    for (int j = 0; j < cols; j++) {
        for (int i = 0; i < rows / 2; i++) {
            temp[i][j] = (image[2.* i][j] + image[2.* i + 1][j]) / sqrt(2.0); // low pass
            temp[i + rows / 2][j] = (image[2.* i][j] - image[2.* i + 1][j]) / sqrt(2.0); // high pass
        }
    }
    image = temp;
}
```

Haar 변환으로 신호를 여러 주파수 밴드로 분해한다. 수평, 수직으로 로우 패스, 하이 패스 필터링을 수행하고 반복한다. 한 번의 반복으로 이미지는 가로, 세로 약 2 개의 주파수 대역으로 분해된다.

- 역 Haar-Wavelet 변환

```
void inverseHaarTransform(vector<vector<double>>& image, int rows, int cols)
{
    vector<vector<double>> temp(rows, vector<double>(cols));
    // vertical inverse transform
    for (int j = 0; j < cols; j++) {
        for (int i = 0; i < rows / 2; i++) {
            temp[2.* i][j] = (image[i][j] + image[i + rows / 2][j]) / sqrt(2.0); // low pass
            temp[2.* i + 1][j] = (image[i][j] - image[i + rows / 2][j]) / sqrt(2.0); // high pass
        }
    }
    image = temp;

    // horizontal inverse transform
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols / 2; j++) {
            temp[i][2.* j] = (image[i][j] + image[i][j + cols / 2]) / sqrt(2.0); // low pass
            temp[i][2.* j + 1] = (image[i][j] - image[i][j + cols / 2]) / sqrt(2.0); // high pass
        }
    }
    image = temp;
}
```

수직, 수평 방향으로 로우 패스 필터로 생성된 계수 값에 하이 패스 필터로 생성된 계수 값을 더하고 빼서 복원을 수행한다.

- 양자화

```
void quantizeByFrequency(vector<vector<double>>& image, int rows, int cols, const vector<int>& quantLevels)
{
    int bands = quantLevels.size();
    int step = rows / (1 << (bands - 1));

    // apply quantization level to each frequency band
    for (int b = 0; b < bands; b++) {
        int start = step * b;
        int end = step * (b + 1);
        double factor = pow(2.0, quantLevels[b]);

        for (int i = start; i < end && i < rows; i++)
            for (int j = start; j < end && j < cols; j++)
                image[i][j] = round(image[i][j] / factor) * factor; // quantize the coefficients
    }
}
```

주파수 대역 별로 다른 수준의 양자화를 적용할 경우 사람이 인지하기 힘든 고주파 영역에 높은 양자화 수준을 적용해 용량을 낮추면서 화질을 유지할 수 있다.

- 프로그램 시퀀스

각 이미지에 대해 Haar 변환을 수준별로 수행하고 양자화 -> 역양자화 -> 역 Haar 변환 순서대로 수행한 뒤 해당 이미지에 대한 MSE 를 출력한다. 그 다음 Haar 변환을 적용하지 않은 이미지에 대해 양자화 -> 역양자화를 수행하고 해당 이미지에 대한 MSE 를 출력한다.

이미지 Lenna_256x256_yuv400_8bit.raw 에 대한 실행 결과는 다음과 같다.

```
Image: Lenna_256x256_yuv400_8bit.raw
-> Output: Lenna_256x256_yuv400_8bit_Haar3_FreqQuant.raw
Haar Iterations: 3
MSE: 1.41318

Image: Lenna_256x256_yuv400_8bit.raw
-> Output: Lenna_256x256_yuv400_8bit_NoHaar_Quant1.raw
No Haar Transform
Quantization Level: 1
MSE: 0.509003

Image: Lenna_256x256_yuv400_8bit.raw
-> Output: Lenna_256x256_yuv400_8bit_NoHaar_Quant2.raw
No Haar Transform
Quantization Level: 2
MSE: 1.45499

Image: Lenna_256x256_yuv400_8bit.raw
-> Output: Lenna_256x256_yuv400_8bit_NoHaar_Quant3.raw
No Haar Transform
Quantization Level: 3
MSE: 5.50308
```

3. 실행 결과

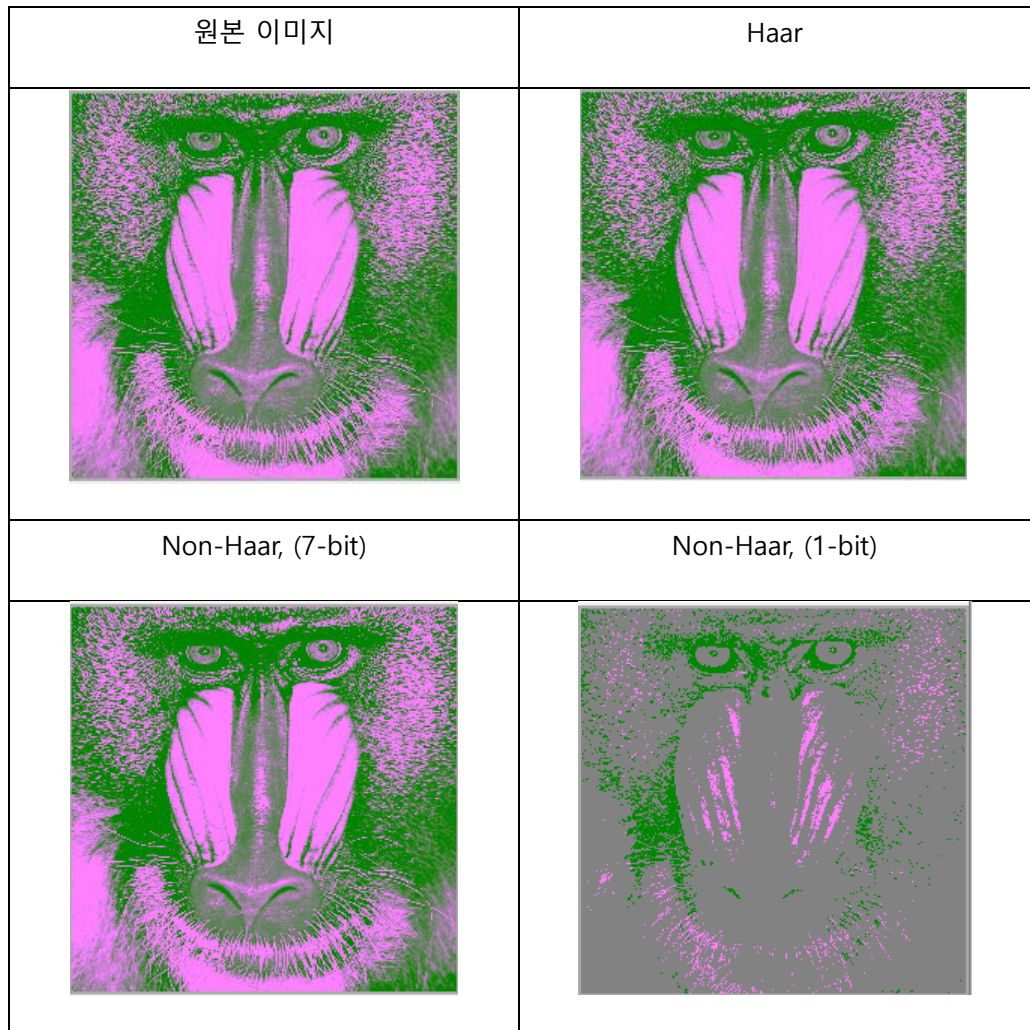
- Airplane_256x256_yuv400_8bit.raw

원본 이미지	Haar
	
Non-Haar, (7-bit)	Non-Haar, (1-bit)
	

MSE

이미지	Haar	7bit	6bit	5bit	4bit	3bit	2bit	1bit
MSE	1.3614	0.4989	1.4939	5.5696	21.5027	95.909	315.169	2033.96

- Baboon_256x256_yuv400_8bit.raw



MSE

이미지	Haar	7bit	6bit	5bit	4bit	3bit	2bit	1bit
MSE	1.3831	0.4981	1.5021	5.5229	21.6289	85.499	305.623	1387.16

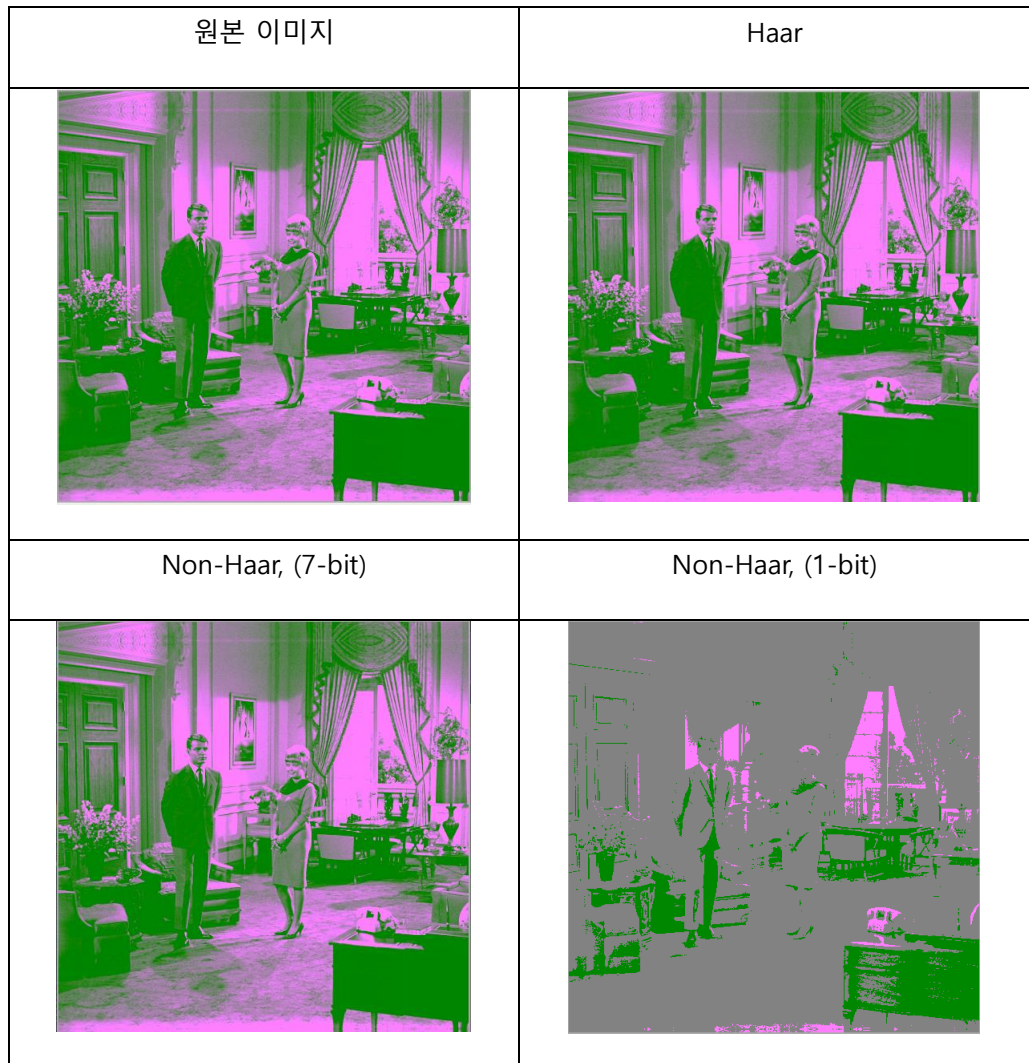
- Barbara_512x512_yuv400_8bit.raw



MSE

이미지	Haar	7bit	6bit	5bit	4bit	3bit	2bit	1bit
MSE	1.3165	0.5006	1.4955	5.5350	21.141	80.9577	398.203	1440.23

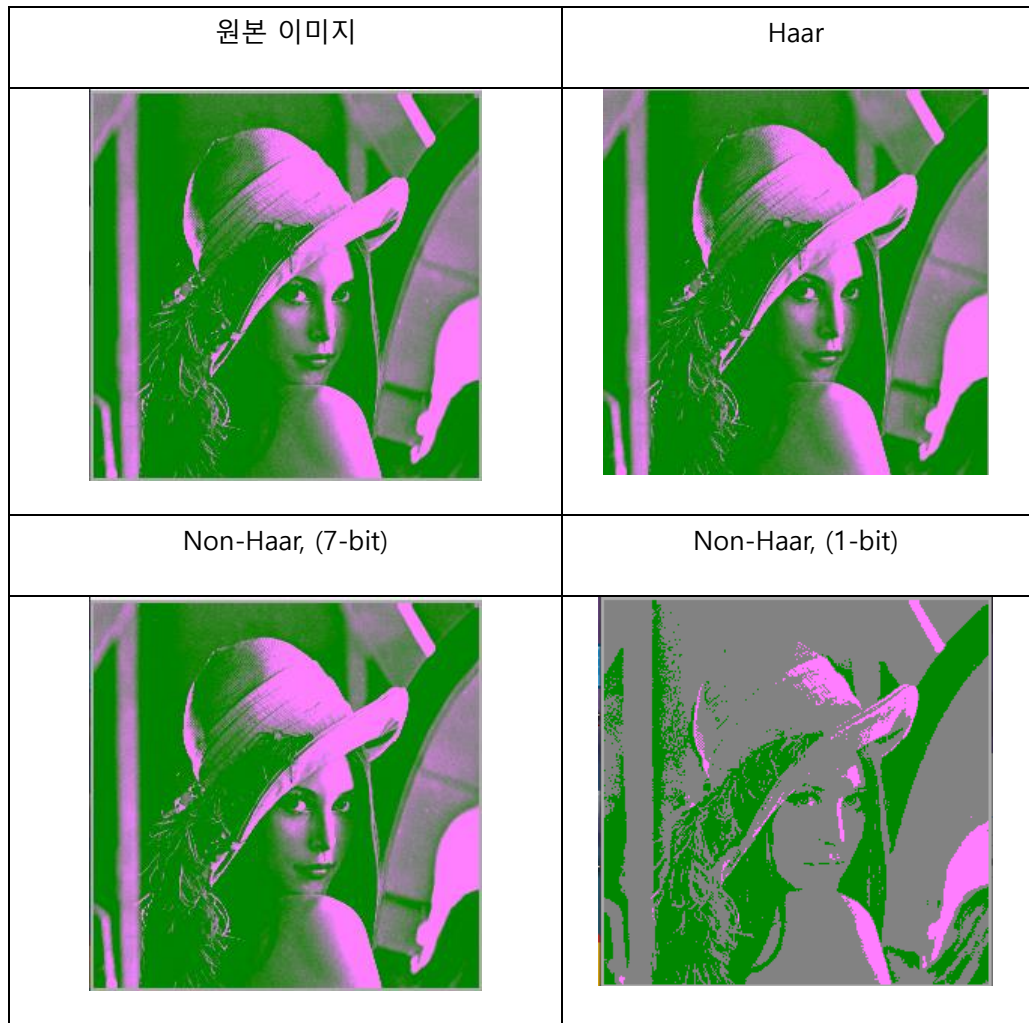
- Couple_512x512_yuv400_8bit.raw



MSE

이미지	Haar	7bit	6bit	5bit	4bit	3bit	2bit	1bit
MSE	1.2970	0.5096	1.4620	5.4979	21.4959	87.144	324.543	1032.12

- Lenna_256x256_yuv400_8bit.raw



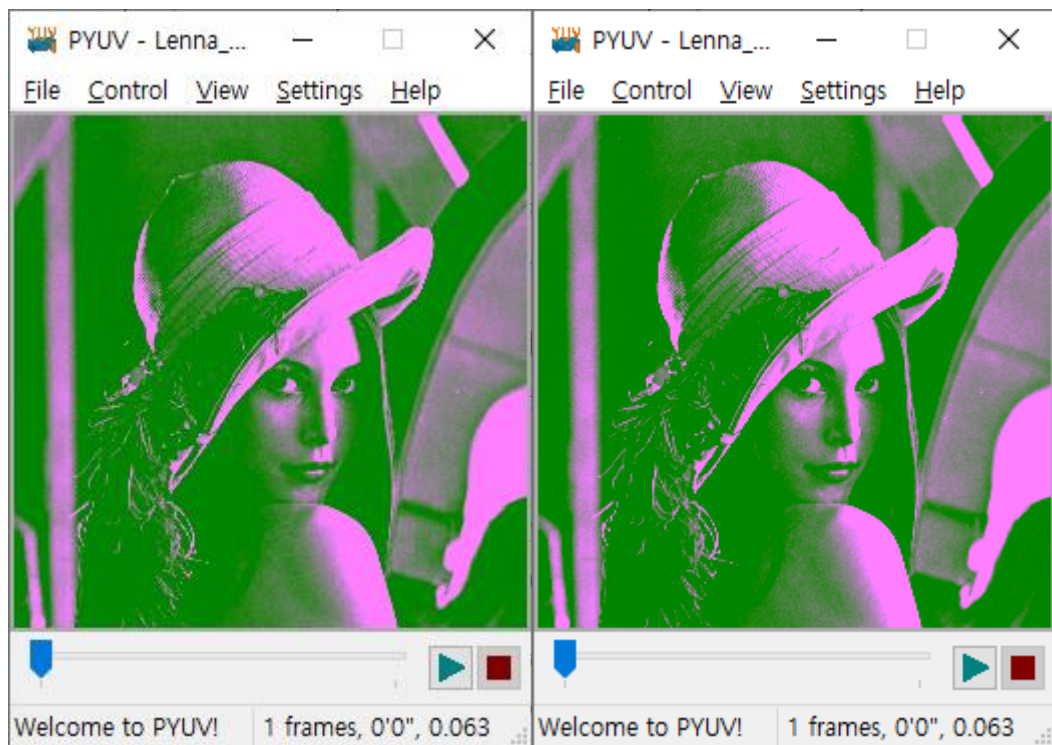
MSE

이미지	Haar	7bit	6bit	5bit	4bit	3bit	2bit	1bit
MSE	1.4131	0.5090	1.4549	5.5030	21.454	83.232	337.103	1379.25

4. 고찰

- 결과 분석 (MSE)

각 이미지에 대한 MSE 를 정리한 도표를 보면 Haar-Wavelet 변환을 수행하고 양자화를 수행한 이미지보다 변환을 수행하지 않고 낮은 수준의 양자화를 수행한 이미지의 MSE 가 더 낮았다. 하지만 두 이미지를 주관적으로 분석했을 땐 화질의 차이가 거의 없었다. 양자화 수준이 높아질수록 화질이 감소하면서 MSE 가 증가했는데, MSE 가 비슷하게 출력된 6 비트 양자화 이미지보다 화질이 근소하게 좋았다.



(좌측이 Haar 변환을 수행한 이미지, 우측이 6 비트 수준으로 양자화 한 이미지이다.)

아마 Haar 변환을 통해 분리된 고주파역에 높은 수준의 양자화를 적용하여 MSE 가 비슷함에도 불구하고 약간 더 좋은 화질을 보여주는 것이라고 생각한다.